PicNEat - Product Requirements Document

Version 1 | March 1, 2026 | Author: Mahek

---

1. Overview

Vision

PicNEat is an iPhone app that lets Purdue students track their calories - by simply taking a photo of their dining hall meal. Long-term, it becomes the gold standard for on-campus nutrition awareness, as accurate and delightful to use as CAL AI.

Problem

Purdue students eating at dining halls have no easy way to track calories. Nutrition info is scattered across PDFs and dining websites. Students either ignore tracking entirely or spend 10+ minutes manually logging - both leading to poor dietary awareness.

Solution

Snap a photo → Groq Vision API (llama-3.2-90b-vision) Vision identifies the food + estimates portion size with high accuracy → calories are auto-mapped via USDA FoodData Central (600k+ foods) with a Purdue dining menu overlay → logged instantly, with a gamified mission system that keeps students coming back daily.

Purdue dining data is scraped daily from dining.purdue.edu/menus - all dishes and their corresponding calories across all dining courts are pulled automatically and stored in Supabase. When a detected food matches a Purdue menu item, the Purdue data takes priority over USDA for maximum campus accuracy.

---

2. Target Users

Primary: Purdue students eating at on-campus dining halls (est. 10,000+ eligible users)

Early adopter profile: Health-conscious students (athletes, fitness-oriented), students with dietary restrictions, CS/engineering students comfortable with new apps

---

3. Goals & Success Metrics

Goal	V1 Metric	V2+ Metric
Food ID accuracy	>85% on any food (Groq)	>95% (custom model)
Calorie estimate accuracy	±15% of true value	±10%
Time to log a meal	<10 seconds	<5 seconds
DAU/MAU ratio	>30%	>60%
User retention (Day 30)	>25%	>50%

---

4. V1 vs V2 — Model Architecture Distinction

This is the most important architectural decision in the product. V1 and V2 are deliberately designed so that swapping the detection model requires **zero changes to the iOS app and zero changes to the database** - only the inference layer in the FastAPI backend is replaced.

V1 — Groq Vision API + USDA (Ship immediately, works on everything)

The V1 detection pipeline uses Groq Vision API (llama-3.2-90b-vision) as the food identification engine, feeding into the USDA FoodData Central API for calorie/macro lookup. This requires no model training, no GPU hosting, and no custom dataset — it works on any food from day one with Cal AI-level accuracy.

```
iPhone Camera
   ↓ image (base64)
FastAPI /analyze-meal endpoint
   ↓
Groq Vision API (llama-3.2-90b-vision) Vision API
   → returns structured JSON:
     [{ name: "grilled chicken breast skinless",
        portion: "120g", confidence: 0.92 }]
   ↓
Purdue Menu DB lookup (fuzzy match first)
   → if match found: use Purdue calorie data
   → if no match: fall through to USDA
   ↓
```

USDA FoodData Central API
→ maps food name + portion → calories, protein, carbs, fats
↓
Response: { foods, total_cals, macros }

Why portion estimation is accurate in V1: Groq Vision has strong spatial reasoning - it estimates portion sizes contextually and returns gram weights directly. Combined with USDA's per-100g nutrient data, this produces accurate calorie estimates without any plate-relative heuristics or depth cameras. The prompt explicitly instructs Groq to use USDA-compatible food names, which maximizes lookup accuracy downstream.

Groq System Prompt (core of V1 accuracy):

You are a world-class registered dietitian and food scientist with 20 years of experience in nutritional analysis and food portion estimation. You have been trained on USDA FoodData Central, the NIH portion size database, and thousands of restaurant and dining hall menus. TASK: Analyze this food image with the precision of a metabolic researcher. Your calorie estimates will be used for health tracking — accuracy is critical. STEP 1 — SCENE ANALYSIS (reason before answering): Before identifying foods, assess the scene: - What is the plate/bowl/container size? (standard dinner plate = 10-11", side plate = 7-8", bowl = 6", to-go container = 8x6") - Are there reference objects visible (fork, cup, hand, phone) to calibrate scale? - What is the plate fullness? (quarter / half / three-quarter / full) - What is the camera angle? (top-down, 45°, side) — adjust depth estimates accordingly - Is food stacked or layered? Estimate hidden volume. STEP 2 — FOOD IDENTIFICATION: For each distinct food item: - Use the most specific USDA FoodData Central name possible ✓ "chicken breast, grilled, skinless" not "chicken" ✓ "rice, white, long-grain, cooked" not "rice" ✓ "broccoli, cooked, boiled, drained, without salt" not "broccoli" - Identify cooking method (raw/boiled/grilled/fried/baked/roasted/steamed) - Identify visible added fats (butter, oil, sauce, dressing, cheese) - Note if food appears to be restaurant/dining-hall style (typically 20-30% more calories than home-cooked due to added fats) STEP 3 — PORTION ESTIMATION: Use ALL of the following methods and cross-reference: - Plate coverage method: what % of a standard plate does this food occupy? - Hand portion method: palm = ~3oz protein, cupped hand = ~½ cup carbs, thumb = ~1oz fat - Visual volume method: estimate length × width × height in cm, apply food density - Reference object method: use any visible utensils, cups, or hands to calibrate - Cooking shrinkage: account for water loss (meat loses ~25% weight when cooked) For mixed dishes (pasta, stir fry, soup): estimate each component separately then sum. STEP 4 — CONFIDENCE CALIBRATION: Be honest about uncertainty. If food is obscured, sauced-over, or ambiguous: - Lower confidence score - Provide the most likely identification - Note what is making identification difficult CRITICAL RULES: 1. NEVER round calories to suspiciously clean numbers — real portions are 247g not 250g 2. Account for ALL visible calories including sauces, dressings, oils, toppings 3. If a dish looks like a dining hall portion, apply the institutional cooking bias (+20% fat) 4. Err toward HIGHER estimates when uncertain — underestimating is more dangerous for health tracking than overestimating 5. Identify cooking vessel residue as calories (oily pan = add ~50-100 cal for absorbed oil)

V2 — Custom Model (Plug-in replacement, no app changes)

In V2, the Groq Vision API (llama-3.2-90b-vision) call is replaced by a fine-tuned on-device CoreML model trained on Purdue dining hall food photos. The swap happens entirely inside the FastAPI backend behind the same `/analyze-meal` endpoint. The iOS app, database schema, and all gamification logic remain 100% unchanged.

```
V1 endpoint:   image → Groq Vision API (llama-3.2-90b-vision-preview) → structured JSON
V2 endpoint:   image → CoreML/YOLOv8 fine-tuned → same structured JSON
                                    ↑
                        identical output schema — app never knows the difference
```

The V2 model is trained on labeled photos collected during the V1 beta period via the user correction flow (tap the match % badge → correct the food ID → this data is logged and used for training).

V2 hosting: On-device CoreML for offline inference. No server costs.

---

5. Technical Architecture

V1 Full Stack

```
iOS App (SwiftUI)
  ↓ image upload (URLSession, multipart)
FastAPI Backend (Railway free tier)
  ├── /analyze-meal     → Groq Vision API (llama-3.2-90b-vision-preview) + USDA lookup
  ├── /log-meal         → write to Supabase meal_logs
  ├── /daily-summary    → read today's meals + totals
  └── /menu-search      → search Purdue menu DB
Supabase (PostgreSQL + Storage)
  ├── users
  ├── meal_logs
  ├── purdue_menu       ← refreshed daily by scraper
  └── user_progress
USDA FoodData Central API (free, 600k+ foods)
Groq Vision API (llama-3.2-90b-vision-preview) — free tier
```

Database Schema

sql

```
users (
```

```
  id uuid PRIMARY KEY,
  purdue_email text UNIQUE,
  calorie_goal int DEFAULT 2000,
  created_at timestamp
)

meal_logs (
  id uuid PRIMARY KEY,
  user_id uuid REFERENCES users,
  photo_url text,
  detected_foods jsonb,   -- [{ name, portion_g, cals, protein, carbs, fats, confidence }]
  total_calories int,
  logged_at timestamp
)

purdue_menu (
  id uuid PRIMARY KEY,
  item_name text,
  dining_hall text,
  calories_per_serving int,
  serving_size_g int,
  category text,
  last_updated date        -- scraper updates this daily
)

user_progress (
  id uuid PRIMARY KEY,
  user_id uuid REFERENCES users,
  fuel_total int DEFAULT 0,
  streak_days int DEFAULT 0,
  last_logged_date date,
  badges_earned jsonb,
  mission_number int DEFAULT 1
)

-- V2 training data collection (built into V1)
food_corrections (
  id uuid PRIMARY KEY,
  meal_log_id uuid REFERENCES meal_logs,
  original_detection text,
  corrected_food text,
  photo_url text,
  created_at timestamp
)
```

Purdue Dining Scraper

Runs daily via a Railway cron job. Scrapes dining.purdue.edu/menus for all dining courts — Wiley, Ford, Hillenbrand, Earhart, and Windsor — pulling every dish name and its corresponding calorie count. Menu data is upserted into `purdue_menu` each morning at 6am before breakfast service.

python
```
# scraper runs daily at 6am
async def scrape_purdue_menus():
    courts = ["wiley", "ford", "hillenbrand", "earhart", "windsor"]
    for court in courts:
        dishes = await fetch_court_menu(court)
        await upsert_purdue_menu(dishes)  # updates last_updated = today
```

Inference

- iOS: SwiftUI, URLSession, PhotosPicker
- Backend: FastAPI on Railway (free tier)
- Database: Supabase (free tier, handles <1500 users)
- Food detection (V1): Groq Vision API (llama-3.2-90b-vision-preview)
- Food detection (V2): CoreML on-device (plug-in replacement)
- Nutrition data: USDA FoodData Central API (free)
- Photo storage: Supabase Storage

---

6. Features

V1 (MVP)

Core Loop:
- In-app camera capture
- Groq Vision API (llama-3.2-90b-vision-preview) Vision food detection with portion estimation
- USDA FoodData Central calorie/macro lookup
- Purdue dining menu overlay (Purdue data takes priority when matched)
- Instant results with bounding box, nutrition card, and match confidence
- Tap-to-correct food ID (correction data feeds V2 training set)
- Daily calorie log

Account & Data
- Purdue email sign-in via Google Auth
- Personal calorie goal setting

- Daily + weekly calorie summary

Gamification
- Space/mission theme with Neil the astronaut as mascot
- "Fuel" currency earned per logged meal
- 7-day streak tracking
- Planet badge progression (Moon → Mercury → Venus → Jupiter → Uranus)
- Duolingo-style popup celebrations on meal log, streak milestone, goal hit, and badge unlock
- Leaderboard showing Purdue friends' streaks

Out of scope for V1*
- Custom trained food model (V2)
- Android
- Apple Watch / widget
- Barcode scanning

V2 - V4

- Custom CoreML model trained on Purdue dining photos (plug-in swap, no app changes)
- Full macro tracking (protein, carbs, fat, fiber)
- Meal history + trend analytics
- Smart suggestions based on remaining daily calories
- Live Purdue dining API sync (replacing scraper if API becomes available)
- Barcode scanning for packaged snacks
- Widget + Apple Watch glanceable progress
- Social layer — friend groups, challenges

---

7. User Flow

```
Open App
  → Home (EAT tab)
      - Daily calorie ring
      - Neil hero card with rocket progress bar
      - Today's meal log
  → Tap "PIC" tab
      - Camera opens with helmet HUD framing
      - Neil is "searching" during viewfinder
  → Take photo
      - Neil switches to "working" animation during 2-4s analysis
  → Results screen
      - Photo with bounding box on detected food
```

- Nutrition overlay card (food name, cals, protein, carbs, fats, match %)
- Tap match % → correction picker (also captures training data)
- [Retake] or [Log Meal]

→ Log Meal tapped
- Neil bursts onto screen → confetti → +FUEL animation (1.5s)
- Auto-returns to EAT tab
- Calorie ring + streak update

→ Daily goal hit
- Full-screen: Neil lands rocket on planet → MISSION COMPLETE banner

→ Streak milestone (Day 3 / 7 / 14 / 30)
- Neil celebration scales with streak length

→ Streak broken
- Neil floats sad in black space → RESUME MISSION button

---

8. UI/UX Design Direction

Design Language

Bold all-caps personality throughout. Permanent macro color system:

- 🟠 Orange → Calories
- 🔴 Red → Protein
- 🟡 Yellow-orange → Carbs
- 🔵 Blue → Fats
- 🟢 Green → Positive actions (Log Meal, confirm)

Neil — The Mascot

Neil is a named character, not a decoration. Every notification, empty state, and celebration speaks *as Neil*. His astronaut image changes contextually across the app:

Moment	Neil state
Home, on track	Sitting on planet, content
Home, under-eaten	Floating hungry in space
Home, goal hit	Dancing / celebrating
PIC viewfinder	Searching / scanning
PIC analyzing	Working at laptop

PIC result	Eating detected food
Streak broken	Floating sad, helmet cracked
Badge unlocked	Landing on new planet
Empty state	Floating, "Houston, no meals yet"

Popup / Celebration System (Duolingo-style)

Five distinct popup moments, each with auto-dismiss timing:

Meal Logged — 1.5s auto-dismiss. Neil bursts in, confetti, +FUEL number slams in large then shrinks. Subtle whoosh sound.

Daily Goal Hit — 3s. Neil's rocket lands on current planet with screen shake and dust clouds. MISSION COMPLETE banner drops. Metallic ding sound.

Streak Milestone — 2.5s. Neil animation escalates with streak: Day 3 = wave, Day 7 = backflip, Day 30 = on fire. Increasingly hype sound effect.

Streak Broken — persistent until dismissed. Black space. Neil floating lifelessly. Cracked helmet. Low drone sound. "Houston... we lost the signal." → RESUME MISSION button.

Badge Unlocked — 4s cinematic. Planet rotates into frame. Neil lands on it. Badge slams down. Stars burst. Most elaborate celebration in the app.

Please find specific design assets: Astronaut assets for popups

Screen Specifications

EAT Tab

- Neil hero card reacts to calorie status (content / hungry / celebrating)
- Rocket progress bar fills as meals are logged toward daily goal
- Calorie goal denominator large and legible
    - LOG MEAL CTA persistent below meal list

PIC Tab *(highest priority screen)*

- Helmet HUD aesthetic on camera viewfinder
- Bounding box on detected food in results photo
- Two buttons only: Retake / Log Meal
- Match % badge tappable → correction flow

LEAD Tab

- Mission progress card → Badges → Leaderboard (top to bottom)
- Next unearned badge shows partial glow at 50% progress
- Leaderboard entries have color avatars

FIND Tab

- Filter pills use macro color system when selected
- Dining hall cards show 2-3 food previews + walking time
- Neil speech bubble changes based on active filter

---

9. Gamification System

Currency: Fuel Users earn Fuel by logging meals, hitting calorie goals, and maintaining streaks. Fuel directly maps to rocket progress toward the next planet.

Reward Loops

Loop	Trigger	Reward
Micro	Log any meal	+Fuel animation, confetti burst
Daily	Hit calorie goal	Streak +1, Neil celebrates
Weekly	7-day streak	Badge progress, mission advance

Planet Badge Progression

Badge	Fuel Required
🌙 Moon	100
🔴 Mercury	200
🟣 Venus	300
🟠 Jupiter	400
🟢 Uranus	500

Locked badges show partial glow at 50% progress. Earned badges pulse briefly when viewed.

---

10. Risks & Mitigations

Risk	Mitigation
Groq Vision API (llama-3.2-90b-vision-preview) misidentifies food	Tap-to-correct UI always available; corrections feed V2 training data
Purdue menu data goes stale	Daily automated scraper at 6am refreshes all dining courts
Low retention after novelty	Duolingo-style popup system + streak guilt loop keeps daily engagement
OpenAI API costs at scale	V2 custom CoreML model eliminates API cost entirely — designed as direct plug-in swap
V1 → V2 model transition risk	Identical JSON output schema means zero iOS changes required

---

11. Build Phases (3-Day Sprint)

Day 1 — Backend + Data Pipeline

- FastAPI project scaffolded on Railway
- Groq Vision API (llama-3.2-90b-vision) `/analyze-meal` endpoint live and tested with food photos
- USDA FoodData Central integration — food name → calories/macros
- Purdue dining scraper built and run once to seed `purdue_menu` table
- Supabase schema deployed (all 5 tables)
- End of Day 1 goal: send a food photo via Postman, receive accurate calorie JSON back

Day 2 — iOS App Core

- SwiftUI project with 4-tab navigation (PIC / EAT / LEAD / FIND)
- PIC tab: camera capture → POST to backend → results screen with bounding box overlay + nutrition card
- EAT tab: daily calorie ring, Neil hero card with rocket progress bar, meal log list
- Supabase Auth: Purdue email sign-in
- End of Day 2 goal: take a photo on iPhone, see calories appear, meal logs to Supabase

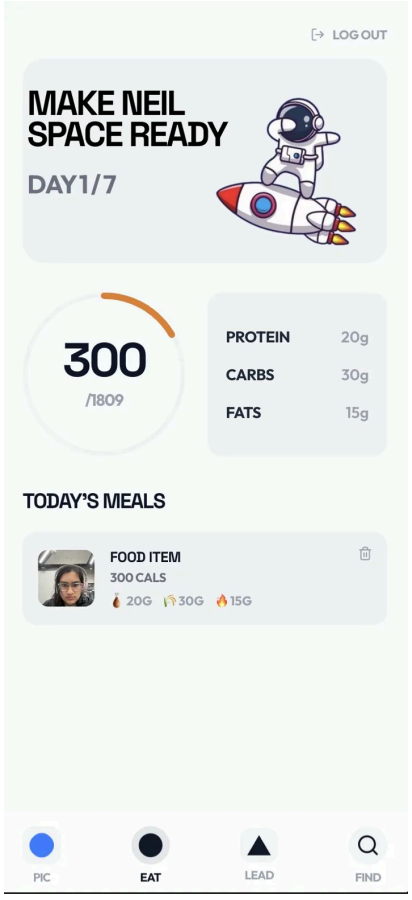
Day 3 — Gamification + Polish

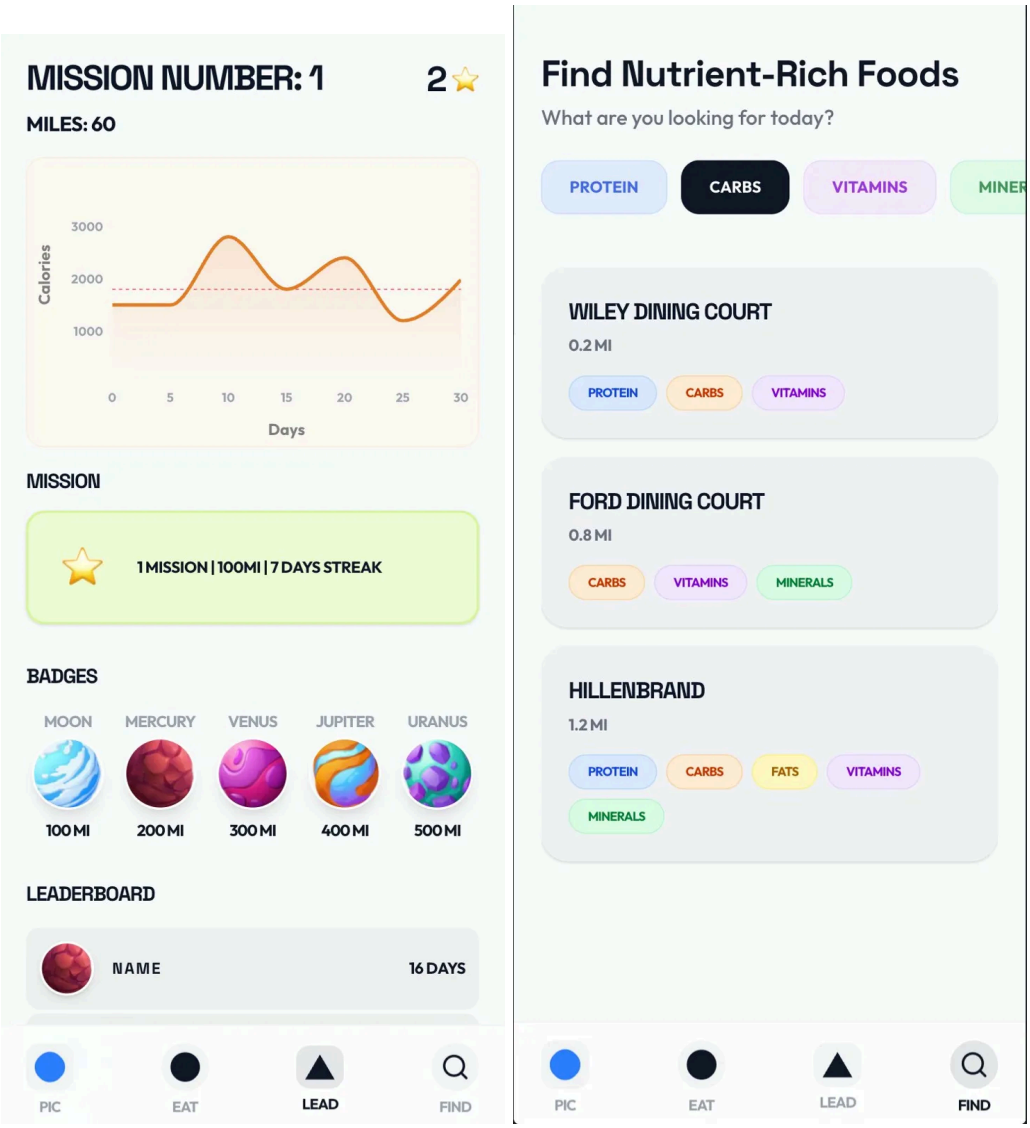
- Fuel system: earn on meal log, persist in `user_progress`
- Streak tracking: daily check-in logic, broken streak detection
- All 5 Neil popup celebrations (meal logged, goal hit, streak milestone, streak broken, badge unlock)
- LEAD tab: mission card, badge progress rings, leaderboard
- FIND tab: dining hall cards with filter pills, food previews, walking time
- TestFlight build submitted to 10 alpha testers
- End of Day 3 goal: fully playable app on TestFlight

---

12. Screenshots of Pilot Version:

Github: https://github.com/mahekparvez/v4picneat





13. API Integration Guide

Environment Variables

Add these to Render's environment config - never hardcode. Kindly create a specific email for the different APIs so it can be accessed by any 3rd person, and provide email ID and password access. This shall be replaced with paid versions of the API key later.

GROQ_API_KEY=

SUPABASE_URL=

SUPABASE_ANON_KEY=.

---

1. Groq Vision — Food Detection

Sign up at console.groq.com, generate a free API key.

- Called once per meal scan at `/analyze-meal`
- Compress image to 800×800px JPEG before sending
- Use model `llama-3.2-90b-vision-preview`, force `response_format: json_object`
- If Groq fails, return "Analysis unavailable" to the app — never crash
- Log every raw response to Supabase before processing

---

2. Open Food Facts — Nutrition Lookup

No signup. No key. Hit directly.

- Called once per detected food item
- `https://world.openfoodfacts.org/cgi/search.pl?search_terms={food_name}&json=1`
- Take `products[0]`, extract `energy-kcal_100g`, `proteins_100g`, `carbohydrates_100g`, `fat_100g`
- Cache every result in Supabase `food_cache` table — never call the API twice for the same food
- If no result, fall back to Groq's own calorie estimate — never return zero to the user

---

3. Purdue Dining Scraper — Campus Menu

No key. Public website.

- Cron job runs daily at 6am ET, scrapes all five courts (Wiley, Ford, Hillenbrand, Earhart, Windsor)
- Click each dish detail page — calories live there, not on the menu index
- Upsert into `purdue_menu`, never delete rows
- If scraper fails, yesterday's data is still valid

Lookup order inside `/analyze-meal`:

1. Fuzzy match → purdue_menu (use if 80%+ match)

2. Check food_cache (use if hit)

3. Call Open Food Facts → cache it (use result)

4. Fall back to Groq estimate (last resort)

---

4. Supabase — Database, Auth, Storage

Sign up at supabase.com, copy URL and anon key into Render.

- Enable Email Auth, create a `meal-photos` storage bucket
- Upload photo to storage before calling Groq — save the URL to `meal_logs` regardless of whether analysis succeeds
- Enable Row Level Security on every table from day one
- Use connection pooler port `6543` not `5432`

V1 → V2 Model Swap

When the custom model is ready, replace the Groq call in `/analyze-meal` with the new model. Output schema stays identical — zero changes needed anywhere else.